

Conversia Intregului intr-un Sir

```
> fun cifra(c)=case c of
#           0 => "0"
#           | 1 => "1"
#           | 2 => "2"
#           | 3 => "3"
#           | 4 => "4"
#           | 5 => "5"
#           | 6 => "6"
#           | 7 => "7"
#           | 8 => "8"
#           | 9 => "9"
#           | e => "Eroare";
val cifra = fn : int -> string
> fun sir(i)=if i<10 then cifra(i)
#           else sir(i div 10)^cifra(i mod 10);
val sir = fn : int -> string
> sir(7);
val it = "7" : string
> sir(123456789);
val it = "123456789" : string
> > sir(~1);
val it = "Eroare" : string
>
```

```

Prelude> :set +m
Prelude> cifra (c) = case c of {
Prelude|             0 -> "0";
Prelude|             1 -> "1";
Prelude|             2 -> "2";
Prelude|             3 -> "3";
Prelude|             4 -> "4";
Prelude|             5 -> "5";
Prelude|             6 -> "6";
Prelude|             7 -> "7";
Prelude|             8 -> "8";
Prelude|             9 -> "9";
Prelude|             _ -> "Eroare"}
Prelude> cifr (7)

<interactive>:14:1:
  * Variable not in scope: cifr :: Integer -> t
  * Perhaps you meant `cifra' (line 2)
Prelude> cifra (7)
"7"
Prelude> sir (i)=if i<10 then cifra (i)
Prelude|         else sir(i `div` 10) ++ cifra(i `mod` 10)
Prelude> sir(7)
"7"
Prelude> sir(123456789)
"123456789"
Prelude> sir( -1)
"Eroare"
Prelude> _

```

Ecuatie de Gradul 1

```
Poly/ML 4.1.3 Release
> exception Ecuatie_Nedeterminata;
exception Ecuatie_Nedeterminata
> exception Ecuatia_Nu_are_Solutie;
exception Ecuatia_Nu_are_Solutie
> fun test x = x<=0.0 andalso x>=0.0;
val test = fn : real -> bool
> test 0.0;
val it = true : bool
> test 1.0;
val it = false : bool
> fun ecgr1(a,b,c)=if (test a) andalso (test b) andalso (test c) then
#                               raise Ecuatie_Nedeterminata
#                               else if (test a) then raise Ecuatia_Nu_are_Solutie
#                               else (c-b)/a;
val ecgr1 = fn : real * real * real -> real
> ecgr1(0.0,0.0,0.0);
Exception- Ecuatie_Nedeterminata raised
> ecgr1(0.0,1.0,0.0);
Exception- Ecuatia_Nu_are_Solutie raised
> ecgr1(2.0,3.0,4.0);
val it = 0.5 : real
> |
```

```

Prelude> :set +m
Prelude> test x = (x<=0) && (x>=0)
Prelude> test 0.0
True
Prelude> test 1.0
False
Prelude> ecgr1(a,b,c)= if (test a) && (test b) && (test c) then error "Ecuatie nedeterminata"
Prelude|         else if (test a) then error "Ecuatia nu are solutie"
Prelude|         else (c-b)/a
Prelude> ecgr1(0.0,0.0,0.0)
*** Exception: Ecuatie nedeterminata
CallStack (from HasCallStack):
  error, called at <interactive>:5:56 in interactive:Ghci4
Prelude> ecgr1(0.0,1.0,0.0)
*** Exception: Ecuatia nu are solutie
CallStack (from HasCallStack):
  error, called at <interactive>:6:38 in interactive:Ghci4
Prelude> ecgr1(2.0,3.0,4.0)
0.5

```

Simplifica O Fractie

```
Poly/ML 4.1.3 Release
> exception Numitor_Nul;
exception Numitor_Nul
> fun cmmdc(m,n)=if m=0 then n
#               else cmmdc(n mod m,m);
val cmmdc = fn : int * int -> int
> cmmdc(60,90);
val it = 30 : int
> fun fract(a,b)=if b=0 then raise Numitor_Nul
#               else let
#                   val comun=cmmdc(a,b)
#                   in
#                       (a div comun,b div comun)
#                   end;
val fract = fn : int * int -> int * int
> fract(60,90);
val it = (2, 3) : int * int
> |
```

```

Prelude> :set +m
Prelude> cmmdc(n,n)=if m==0 then n
Prelude|         else cmmdc(n `mod` m,n)
Prelude> cmmdc(60,90)
30
Prelude> fract(a,b)=if b == 0 then error "Numitor nul"
Prelude|         else let
Prelude|             comun=cmmdc(a,b)
Prelude|             in
Prelude|             (a `div` comun,b `div` comun)
Prelude> fract(60,90)
(2,3)
Prelude> fract(10,0)
*** Exception: Numitor nul

```

Numere Prime

```
> exception N_Este_prema_Mic;
exception N_Este_prema_Mic
> fun NrPrim1(n,i)= if i > (n div 2) then true
#                       else if n mod i = 0 then false
#                       else NrPrim1(n,i+2);
val NrPrim1 = fn : int * int -> bool
> NrPrim1(3,3);
val it = true : bool
> NrPrim1(5,3);
val it = true : bool
> NrPrim1(9,3);
val it = false : bool
> NrPrim1(123,3);
val it = false : bool
> NrPrim1(127,3);
val it = true : bool
> fun NrPrim(n)=if n<2 then raise N_Este_prema_Mic
#               else if n=2 then "2 este numar prim"
#               else if n mod 2 =0 then "n nu este numar prim"
#               else if NrPrim1(n,3) then "n este numar prim"
#               else "n nu este numar prim";
val NrPrim = fn : int -> string
```

```

> fun NrPrim(n)=if n<2 then raise N_Este_prea_Mic
#           else if n=2 then "2 este numar prim"
#           else if n mod 2 =0 then "n nu este numar prim"
#           else if NrPrim1(n,3) then "n este numar prim"
#           else "n nu este numar prim";
val NrPrim = fn : int -> string
> NrPrim(2);
val it = "2 este numar prim" : string
> NrPrim(3);
val it = "n este numar prim" : string
> NrPrim(4);
val it = "n nu este numar prim" : string
> NrPrim(5);
val it = "n este numar prim" : string
> NrPrim(9);
val it = "n nu este numar prim" : string
> NrPrim(123);
val it = "n nu este numar prim" : string
> NrPrim(127);
val it = "n este numar prim" : string
> NrPrim(1);
Exception- N_Este_prea_Mic raised
> |

```



```

Prelude> :set +m
Prelude> nrPrim1(n,i)= if i > (n `div` 2) then True
Prelude|                     else if (n `mod` i) == 0 then False
Prelude|                     else nrPrim1(n,i+2)
Prelude> nrPrim1(3,3)
True
Prelude> nrPrim1(5,3)
True
Prelude> nrPrim1(9,3)
False
Prelude> nrPrim1(123,3)
False
Prelude> nrPrim1(127,3)
True
Prelude> nrPrim(n)=if n<2 then error "n este prea mic"
Prelude|                     else if n==2 then "2 este un numar prim"
Prelude|                     else if (n `mod` 2) == 0 then "n nu este numar prim"
Prelude|                     else if nrPrim1(n,3) then "n este numar prim"
Prelude|                     else "n nu este un numar prim"
Prelude> nrPrim(1)
*** Exception: n este prea mic
CallStack (from HasCallStack):
  error, called at <interactive>:10:23 in interactive:Ghci7
Prelude> nrPrim(z)

<interactive>:16:8: Variable not in scope: z :: Integer
Prelude> nrPrim(2)
"2 este un numar prim"
Prelude> nrPrim(3)
"n este numar prim"
Prelude> nrPrim(4)
"n nu este numar prim"
Prelude> nrPrim(5)
"n este numar prim"
Prelude> nrPrim(9)
"n nu este un numar prim"
Prelude> nrPrim(123)
"n nu este un numar prim"
Prelude> nrPrim(127)
"n este numar prim"
Prelude> _

```


An Bisect

```
Poly/ML 4.1.3 Release
> exception AnEronat;
exception AnEronat
> fun bisect an = if an < 0 orelse an > 4900 then raise AnEronat
#               else if an mod 4 = 0 andalso an mod 100 <> 0 orelse
#               an mod 400 = 0 then "Anul este bisect"
#               else "Anul nu este bisect";
val bisect = fn : int -> string
> bisect 2018;
val it = "Anul nu este bisect" : string
> bisect 2020;
val it = "Anul este bisect" : string
> bisect 5000;
Exception- AnEronat raised
> bisect 2100;
val it = "Anul nu este bisect" : string
> bisect 2400;
val it = "Anul este bisect" : string
>
```

```
Prelude> :set +1
Prelude> bisect an = if an < 0 || an > 4000 then error "An eronat"
Prelude>      else if an `mod` 4 == 0 && an `mod` 100 /= 0 || an `mod` 400 == 0 then
Prelude>      "Anul este bisect"
Prelude>      else "Anul nu este bisect"
Prelude> bisect 2018
"Anul nu este bisect"
Prelude> bisect 2020
"Anul este bisect"
Prelude> bisect 5000
*** Exception: An eronat
CallStack (from HasCallStack):
  error, called at <interactive>:4:41 in interactive:Ghci1
Prelude> bisect 2100
"Anul nu este bisect"
Prelude> bisect 2400
"Anul este bisect"
Prelude> 
```

Numere Complexe

```
Poly/ML 4.1.3 Release
> infix 3 ++;
infix 3 ++
> fun (r1:real,i1:real)++(r2,i2)=(r1+r2,i1+i2);
val ++ = fn : (real * real) * (real * real) -> real * real
> (1.0,2.0)++(3.0,4.0);
val it = (4.0, 6.0) : real * real
> infix 3 ###;
infix 3 ###
> fun (r1:real,i1:real)###(r2,i2)=(r1-r2,i1-i2);
val ### = fn : (real * real) * (real * real) -> real * real
> (1.0,2.0)###(3.0,4.0);
val it = (~2.0, ~2.0) : real * real
> infix 4 ***;
infix 4 ***
> fun (r1:real,i1:real)***(r2,i2)=(r1*r2-i1*i2,r1*i2+r2*i1);
val *** = fn : (real * real) * (real * real) -> real * real
> (1.0,2.0)***(3.0,4.0);
val it = (~5.0, 10.0) : real * real
> infix 2 ===;
infix 2 ===
> > fun (r1:real,i1:real)===(r2,i2)= (r1<=r2 andalso r1>=r2) andalso
#                                     (i1<=i2 andalso i1>=i2);
val === = fn : (real * real) * (real * real) -> bool
> (1.0,2.0)===(1.0,2.0);
val it = true : bool
> (1.0,2.0)===(1.0,1.0);
val it = false : bool
> (3.0,4.0)***(5.0,6.0);
val it = (~9.0, 38.0) : real * real
> (3.0,4.0)***(5.0,6.0)++(1.0,2.0);
val it = (~8.0, 40.0) : real * real
> (1.0,2.0)++(3.0,4.0)***(5.0,6.0);
val it = (~8.0, 40.0) : real * real
>
```

```

Prelude> :set +n
Prelude> (r1,i1)++-(r2,i2)=(r1+r2,i1+i2)
Prelude> (1.0,2.0)++(3.0,4.0)
(4.0,6.0)
Prelude> (r1,i1)###(r2,i2)=(r1-r2,i1-i2)
Prelude> (1.0,2.0)###(3.0,4.0)
(-2.0,-2.0)
Prelude> (r1,i1)^^(r2,i2)=(r1^r2-i1^i2,r1^i2+r2^i1)
Prelude> (1.0,2.0)^^ (3.0,4.0)
(-5.0,10.0)
Prelude> (r1,i1)===(r2,i2)=(r1==r2) && (i1==i2)
Prelude> (1.0,2.0)===(1.0,2.0)
True
Prelude> (1.0,2.0)===(1.0,1.0)
False
Prelude> (1.0,2.0)+++((3.0,4.0)^^ (5.0,6.0))
(-8.0,40.0)
Prelude>

```

Funcția fibonacci

```
Poly/ML 4.1.3 Release
> exception N_Mai_Mic_decat_1;
exception N_Mai_Mic_decat_1
> fun fibonacci n = if n<1 then raise N_Mai_Mic_decat_1
#                       else if n=1 then 1
#                       else if n=2 then 1
#                       else fibonacci(n-2)+fibonacci(n-1);
val fibonacci = fn : int -> int
> fibonacci 0;
Exception- N_Mai_Mic_decat_1 raised
> fibonacci 1;
val it = 1 : int
> fibonacci 2;
val it = 1 : int
> fibonacci 3;
val it = 2 : int
> fibonacci 10;
val it = 55 : int
```

```

GHCi, version 8.4.3: http://www.haskell.org/ghc/  :? for help
Prelude> :set +m
Prelude> fibonacci n = if n < 1 then error "N este mai mic decat 1"
Prelude|                 else if n==1 then 1
Prelude|                 else if n==2 then 1
Prelude|                 else fibonacci(n-2)+fibonacci(n-1)
Prelude> fibonacci 0
*** Exception: N este mai mic decat 1
CallStack (from HasCallStack):
  error, called at <interactive>:2:29 in interactive:Ghci1
Prelude> fibonacci 1
1
Prelude> fibonacci 2
1
Prelude> fibonacci 3
2
Prelude> fibonacci 10
55
Prelude>

```


Ridicare la Putere

```
Poly/ML 4.1.3 Release
> exception ExponentulEsteNegativ;
exception ExponentulEsteNegativ
> exception BazaNuEstePozitiva;
exception BazaNuEstePozitiva
> infix 9 **;
infix 9 **
> fun a ** b = if b < 0 then raise ExponentulEsteNegativ
#               else if a < 1 then raise BazaNuEstePozitiva
#               else if b = 0 then 1
#               else a * a ** (b - 1);
val ** = fn : int * int -> int
> 10 ** (~1);
Exception- ExponentulEsteNegativ raised
> 0 ** 10;
Exception- BazaNuEstePozitiva raised
> 10 ** 2;
val it = 100 : int
> 2 ** 10;
val it = 1024 : int
> 10 ** 10;
val it = 10000000000 : int
> |
```

```

Prelude> :set +m
Prelude> a ** b = if b<0 then error "Exponentul negativ"
Prelude|         else if a<1 then error "Baza nu este pozitiva"
Prelude|         else if b==0 then 1
Prelude|         else a*(a**(b-1))
Prelude> 10 ** (-1)
*** Exception: Exponentul negativ
CallStack (from HasCallStack):
  error, called at <interactive>:2:22 in interactive:Ghci1
Prelude> 0 ** 10
*** Exception: Baza nu este pozitiva
CallStack (from HasCallStack):
  error, called at <interactive>:3:27 in interactive:Ghci1
Prelude> 10 ** 2

```

100

```
Prelude> 2 ** 10
```

1024

```
Prelude> 10 ** 10
```

10000000000

```
Prelude>
```

Permutări, Aranjamente și Combinari

```
Poly/ML
File Edit Run Help
Poly/ML 4.1.3 Release
> exception N_sau_K_sunt_Negativi;
exception N_sau_K_sunt_Negativi
> exception K_este_Negativ_sau_Nul;
exception K_este_Negativ_sau_Nul
> exception K_este_Mai_Mare_decat_N;
exception K_este_Mai_Mare_decat_N
> fun permutari n = if n < 0 then raise N_sau_K_sunt_Negativi
#               else if n = 0 then 1
#               else n * permutari (n - 1);
val permutari = fn : int -> int
> permutari ~1;
Exception- N_sau_K_sunt_Negativi raised
> permutari 0;
val it = 1 : int
> permutari 4;
val it = 24 : int
> permutari 10;
val it = 3628800 : int
> fun aranjamente(n,k) = if k <= 0 then raise K_este_Negativ_sau_Nul
#               else if k > n then raise K_este_Mai_Mare_decat_N
#               else (permutari n) div (permutari (n - k));
val aranjamente = fn : int * int -> int
> aranjamente(10,0);
Exception- K_este_Negativ_sau_Nul raised
> aranjamente(10,11);
Exception- K_este_Mai_Mare_decat_N raised
```

```

Poly/ML
File Edit Run Help
#           else if k>n then raise K_este_Mai_Mare_decat_N
#           else (permutari n) div (permutari(n-k));
val aranjamente = fn : int * int -> int
> aranjamente(10,0);
Exception- K_este_Negativ_sau_Nul raised
> aranjamente(10,11);
Exception- K_este_Mai_Mare_decat_N raised
> aranjamente(10,1);
val it = 10 : int
> aranjamente(10,5);
val it = 30240 : int
> aranjamente(10,10);
val it = 3628800 : int
> fun combinari(n,k)=if k<0 then raise N_sau_K_sunt_Negativi
#           else if k=0 then 1
#           else aranjamente(n,k) div (permutari k);
val combinari = fn : int * int -> int
> combinari (10, ~1);
Exception- N_sau_K_sunt_Negativi raised
> combinari(10,0);
val it = 1 : int
> combinari(10,1);
val it = 10 : int
> combinari(10,5);
val it = 252 : int
> combinari(10,10);
val it = 1 : int
>

```

Permutări, aranjamente și Combinări

```
λ GHCi
GHCi, version 8.4.3: http://www.haskell.org/ghc/  :? for help
Prelude> :set +m
Prelude> :set +t
Prelude> permutari n = if n<0 then error "n este negativ"
Prelude|                 else if n==0 then 1
Prelude|                 else n*permutari(n-1)
permutari :: (Ord p, Num p) => p -> p
Prelude> permutari (-1)
*** Exception: n este negativ
CallStack (from HasCallStack):
  error, called at <interactive>:3:27 in interactive:Ghci1
Prelude> permutari 0
1
it :: (Ord p, Num p) => p
Prelude> permutari 4
24
it :: (Ord p, Num p) => p
Prelude> permutari 10
3628800
it :: (Ord p, Num p) => p
Prelude> aranjamente(n,k)=if k<=0 then error "k este negativ sau nul"
Prelude|                 else if k>n then error "k este mai mare decat n"
Prelude|                 else (permutari n) `div` (permutari (n-k))
aranjamente :: Integral p => (p, p) -> p
Prelude> aranjamente(10,0)
*** Exception: k este negativ sau nul
CallStack (from HasCallStack):
  error, called at <interactive>:10:31 in interactive:Ghci5
```

λ GHCi

```
error, called at <interactive>:11:35 in interactive:Ghci5
Prelude> aranjamente(10,1)
10
it :: Integral p => p
Prelude> aranjamente(10,5)
30240
it :: Integral p => p
Prelude> aranjamente(10,10)
3628800
it :: Integral p => p
Prelude> combinari(n,k)=if k<0 then error "k este negativ"
Prelude|           else if k==0 then 1
Prelude|           else aranjamente(n,k) `div` (permutari k)
combinari :: Integral p => (p, p) -> p
Prelude> combinari(10, (-1))
*** Exception: k este negativ
CallStack (from HasCallStack):
  error, called at <interactive>:18:28 in interactive:Ghci9
Prelude> combinari(10,0)
1
it :: Integral p => p
Prelude> combinari(10,5)
252
it :: Integral p => p
Prelude> combinari(10,10)
1
it :: Integral p => p
Prelude>
```

Suma Numerelor Pare mai mici decât n

Poly/ML 4.1.3 Release

```
> exception N_este_Prea_Mic;  
exception N_este_Prea_Mic  
> fun sumaPare1(n,s,p)=if p<=n then sumaPare1(n,s+p,p+2)  
#           else s;  
val sumaPare1 = fn : int * int * int -> int  
> fun sumaPare n = if n<2 then raise N_este_Prea_Mic  
#           else sumaPare1(n,0,2);  
val sumaPare = fn : int -> int  
> sumaPare 5;  
val it = 6 : int  
> sumaPare 10;  
val it = 30 : int  
>
```


 GHCi

```
GHCi, version 8.4.3: http://www.haskell.org/ghc/  :? for help
Prelude> :set +m
Prelude> :set +t
Prelude> sumaPare1(n,s,p)=if p<=n then sumaPare1(n,s+p,p+2)
Prelude|                                     else s
sumaPare1 :: (Ord p, Num p) => (p, p, p) -> p
Prelude> sumaPare n = if n<2 then error "N este Prea Mic"
Prelude|                                     else sumaPare1(n,0,2)
sumaPare :: (Ord p, Num p) => p -> p
Prelude> sumaPare 5
6
it :: (Ord p, Num p) => p
Prelude> sumaPare 10
30
it :: (Ord p, Num p) => p
Prelude> -
```

